

**SUSTAINABLE
ENGINEERING
DESIGN APR'25**

*DATE: 12TH JULY 2025
TEAM: GROUP 2*

Table of Contents

1.0 Introduction.....	3
2.0 Applications of CDIO Tools and Techniques.....	3
2.1 Conceive	3
2.2 Design	4
2.3 Implement	7
2.4 Operate.....	9
3.0 Discussion.....	12
3.1 Stakeholder Engagement	13
3.2 Evaluation Against Sustainability (Environmental, Economic and Social Pillars), Safety & Health.....	13
4.0 Conclusion	14
5.0 Appendix.....	15 - 37

1.0 Introduction

The City of Sunway has a strong urban infrastructure with universities, shopping malls, and housing areas. Because of this development, the City of Sunway has a high number of people using its services each day, which includes the number of students at the universities. Therefore, because of the cities' many activities, it generates a large amount of solid waste every day.

One problem, for example, is that the waste generated is often disposed of incorrectly (i.e., mixed), which reduces recycling efficacy, increases the use of landfills, and puts a greater burden on waste disposal systems. As urban areas grow, there is an increasing need to develop smarter, more systematic approaches to waste management. Therefore, our project addresses the problems faced by the City of Sunway by developing a smart waste sorting system that will increase waste separation efficiency and promote more effective waste management.

2.0 Applications of CDIO Tools and Techniques

2.1 Conceive

In the Conceive phase, we brainstormed about different types of problems and their respective solutions in Sunway City such that our initial idea was towards the smart shading system along the canopy walk while our final idea was to develop a smart waste sorting solution for waste management in Sunway City. To choose this final problem statement wisely, we used decision matrices based on the three pillars of sustainability: environmental, social, and economic. The decision matrix for problem selection is in the appendix section.

The waste management using the smart waste sorting system ranked the highest due to its relevance with the UNSDGs 9, 11, 12 and 13 as it shapes an innovative and sustainable city, promotes responsible consumption and supports climate action while also aligning with the Grand Challenges to enhance urban infrastructure and reducing carbon emissions.

2.2 Design

2.2.1 Design approach

In this design phase of our project, we focused on identifying the most suitable solution and components to solve and address the problem identified in Sunway City which was found to be waste management by considering different factors, such as environmental impact, social acceptance, economic feasibility and practical constraints. These factors are aligned with the consideration of making Sunway City more sustainable. In addition to the standard sustainability factor of environmental-social-economic criteria, our prototype development and design approaches are also based on specifically limited project duration, ease of fabrication, and build-time-to-performance ratio. As adopted in choosing our problem in the conceive phase, we will also be using the decision matrix method to ensure a transparent and logical selection process for our solutions.

2.2.2 Choosing of Solution based on Decision Matrix

Based on our group discussion, we came down to four potential solutions, which were automated waste sorter (IOT + ML + actuators), awareness campaign with signage, on-site manual sorting facility (centralized manual sorting) and passive mechanical sorter (non-ML with pure sensors, actuators or diverters). Each of these solutions were then rated using decision matrix method which was scored from 1 to 5 against nine weighted criteria under the standard 3 sustainability categories which were environmental, social and economic. The decision matrix can be seen on the table in the appendix section.

According to the table in the appendix, the automated waste sorter (IoT + ML + actuators) achieved the highest overall score of 3.993 / 5; hence it was selected as the one we will be doing for our prototype.

2.2.3 Sorting Mechanism

The conveyor belt system achieved the highest score of 3.95/5 and was selected as the final sorting mechanism to be used in the prototype, as in general the conveyor mechanism have higher operational reliability, a favorable functionality-to-build ratio, and can be easily integrated with machine learning-based vision system which is what we will be using and this enables real-time classification therefore it has high reliability.

2.2.4 Chassis Construction Material for 3D Printing

PLA achieved the highest overall score of 4.80/5 and was selected to be our chassis material for 3D printing for the final prototype. This is due to PLA's excellent print reliability and dimensional accuracy, faster printing speed, better environmental sustainability and especially lower material cost and greater availability.

2.2.5 Microcontroller Selection

ESP 32 achieved the highest overall score of 4.2/5 and was selected as the microcontroller used for the prototype. The general evaluation on the ESP 32 is that the ESP 32 offers integrated WIFI and Bluetooth which would be perfect for our prototype as we are doing something with IoT, it also has sufficient processing power for basic sensor handling and general logic processing, but most importantly it is compact and has low power consumption.

2.2.6 IoT platform

Blynk achieved the highest overall score of 4.8/5 and was selected as the IoT platform for the project. Blynk provides user-friendly interface for mobile and desktop control applications, it is also very easy to integrate with ESP32, has very well made sufficient real-time monitoring and data visualization, as well as minimal setup time, which is very important for us as we have limited project duration, enabling rapid deployment.

2.2.7 Prototyping Circuit Board Selection

The Donut Board was chosen as the prototyping board, as it offers high reliability and moderate flexibility, while the solderless breadboard also scored the same score, are highly flexible and fast to assemble, their connections can be unstable over time, especially in our project, as the prototype will be put under mechanical vibrations from the motor and also servos. Custom PCB are out of the question, as due to time limitation, it would be hard for us to deal with shipments time, as we would have to potentially order from overseas.

2.2.8 Recognition Intelligence of the System (sensing)

The Image recognition model (ML) achieved the highest overall score of 4.45/5 and was selected as the intelligence recognition system of our prototype. This Image recognition with ML concept has the best combination of accuracy, scalability and integration, as it has superior object classification accuracy that can be made better which makes it scalable and very good implementation compatibility with our system.

2.2.9 AI model for image recognition

YOLO model was selected as the AI model as it scored the highest score of 4.85 out of all the others. This is due to its high detection accuracy for multiple different types of objects in real-time, fast inferencing speed which is perfect for a fast system like our prototype, easy adaptability in terms of modifying the dataset and training and very modular.

2.3 Implement

To implement the chosen design based in section 2.2, the software *SOLIDWORKS* is used to create the 3D model of the mechanical system, and the generated STL file is then passed to the *BambuLab X1C Carbon* for 3D printing, as according to the previous section, the chosen material for this prototype is PLA. Figure 2.3.1 shows the 3D model of the mechanical system.

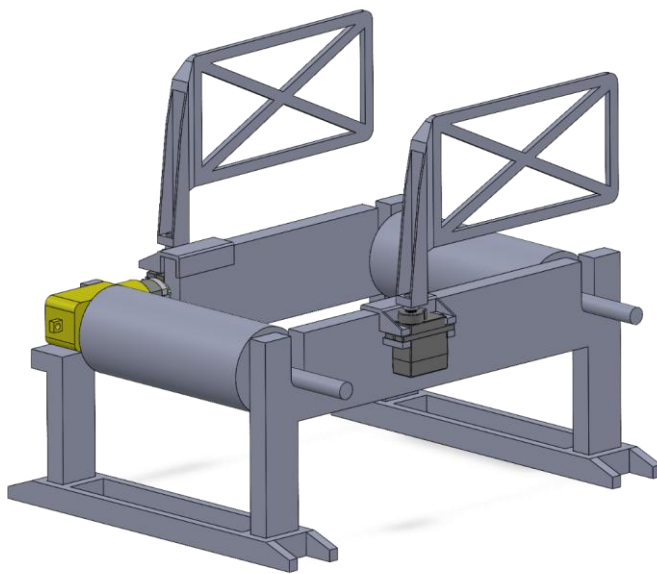


Figure 2.3.1. 3D Model of Mechanical System

The mechanical system consists of 5 major components, beams, drums, feet support, the motor brackets and the diverter arms. When assembled, the system takes approximately **23cm × 18cm × 18.7cm (L × W × H)**, and the total accumulated weight of PLA material used when printing this model is listed in Table 2.3.2, all components are printed with the *Rectilinear* sparse infill pattern at 10% density.

Component	Weight Including Support (grams)
Conveyor Drums	73.46
Conveyor Beams	89.36
Conveyor Supports	31.82
Motor Brackets	6.65
Servo Diverter Arm	40.91
Total	242.2

Table 2.3.2

For the programming of the digital system, the *Arduino IDE* is used to create, compile and upload the code to the ESP32, the code for said microcontroller can be seen in **Appendix 2.3.1**.

For the object identification model, the *YOLO* training algorithm is used to train the model, and a separate python script is used to run the inference engine and produce a signal for the ESP32 depending on the material detected. The training data is a set of 110 images consisting of multiple objects (Metal Cans, Paper Cups and Plastic Bottles) with different shapes, at different orientations and different backgrounds to increase recognition accuracy. The python script that is used to run the inference engine can be seen in **Appendix 2.3.2**.

After detection and sorting of the material waste, the weight of the waste is then uploaded to *BLYNK* Software for data tracking purposes as shown in Figure 2.3.3.

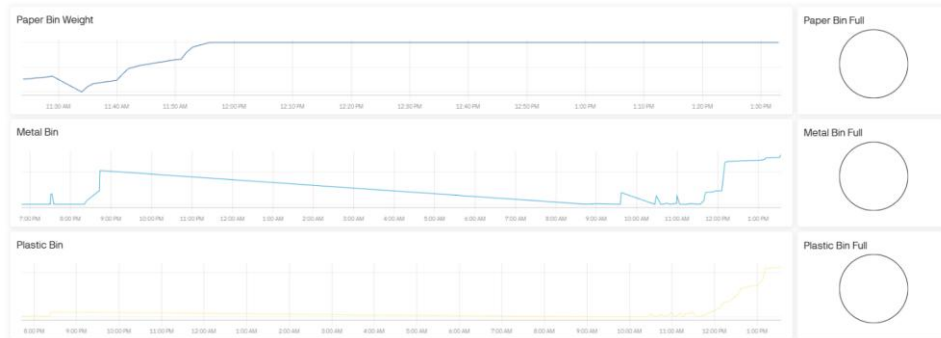


Figure 2.3.3

The physical circuit is built upon the donut perf board and other components as decided by the decision matrix in the previous section, appendix 2.3.4 shows the circuit diagram.

2.4 Operate

The library that allows us to train the YOLO model (Ultralytics) also allows us to generate performance graphs of the object identification model to determine the model performance, below are the performance figures for the trained model, which includes the F1 Curve, Precision Curve, Recall Curve, Precision vs Recall Curve, Confusion Matrix etc, the model is **trained and tested on a 110 images dataset at a ratio of 9:1 for training and validation at 50 epochs:**

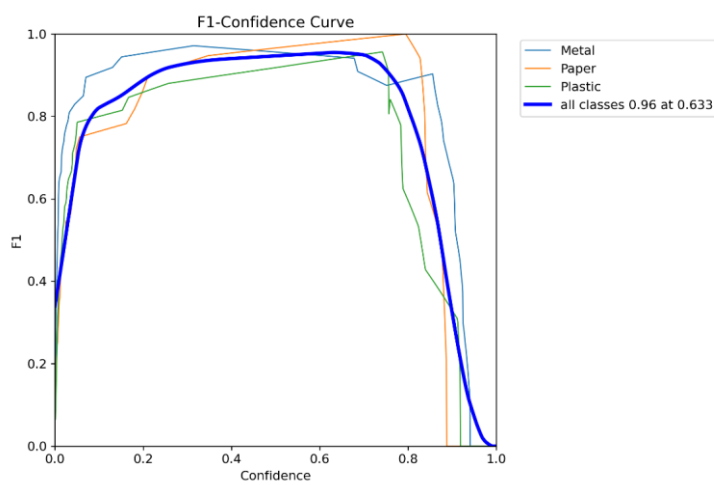


Figure 2.4.1. F1 Curve

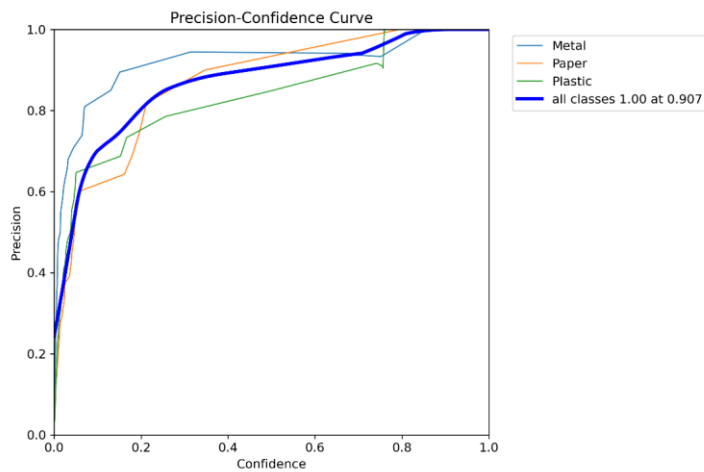


Figure 2.4.2. Precision Curve

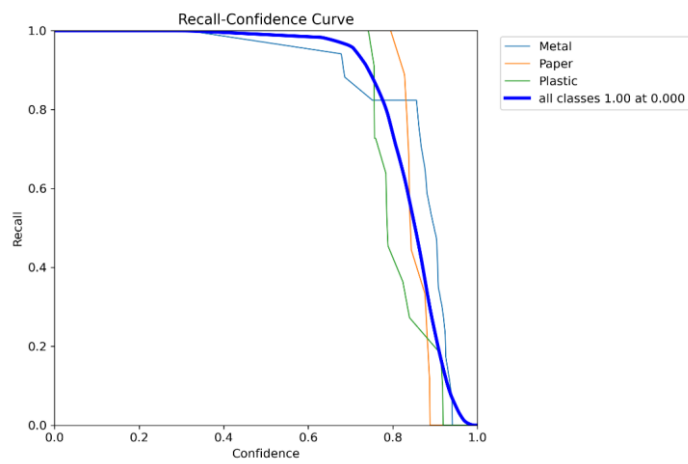


Figure 2.4.3. Recall Curve

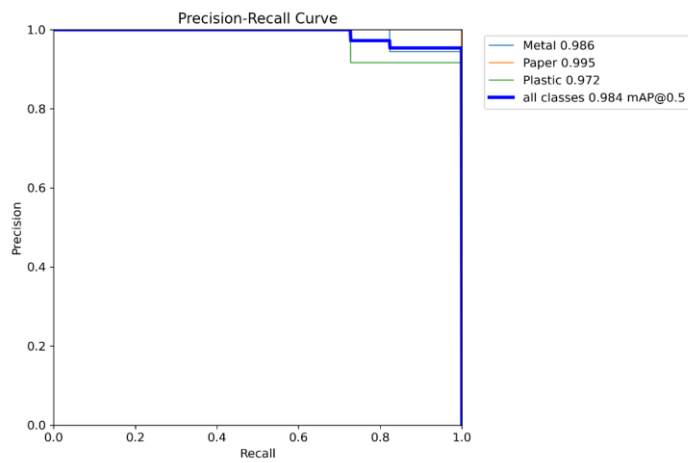


Figure 2.4.4. Precision vs Recall Curve

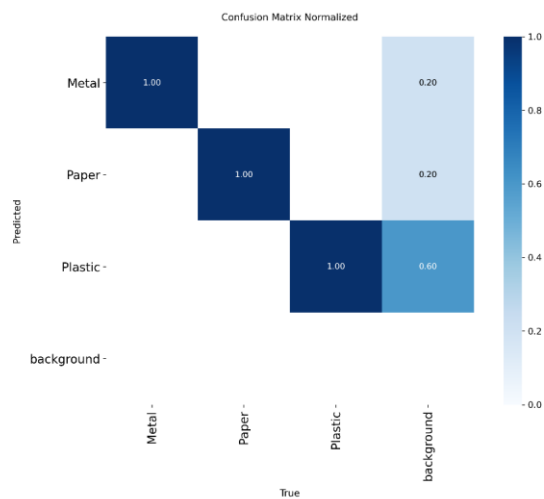


Figure 2.4.5. Normalized Confusion Matrix

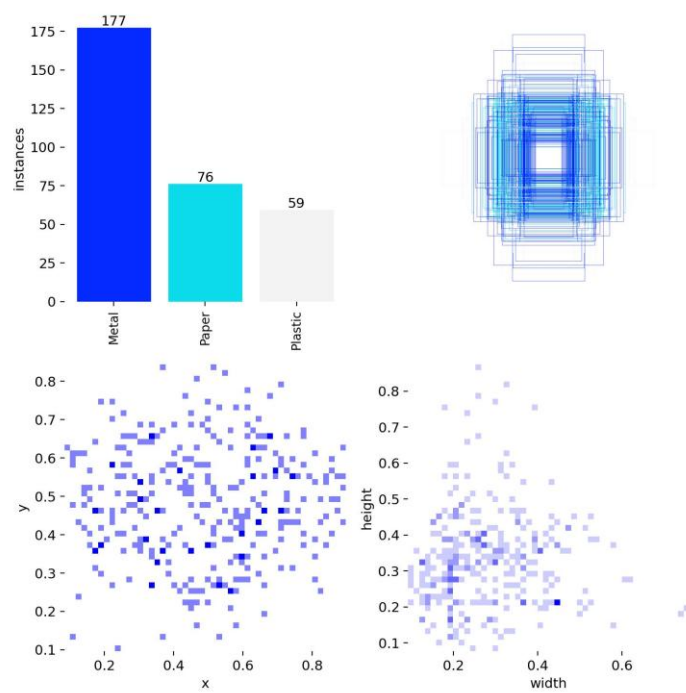


Figure 2.4.6. Visual Summary of The Dataset

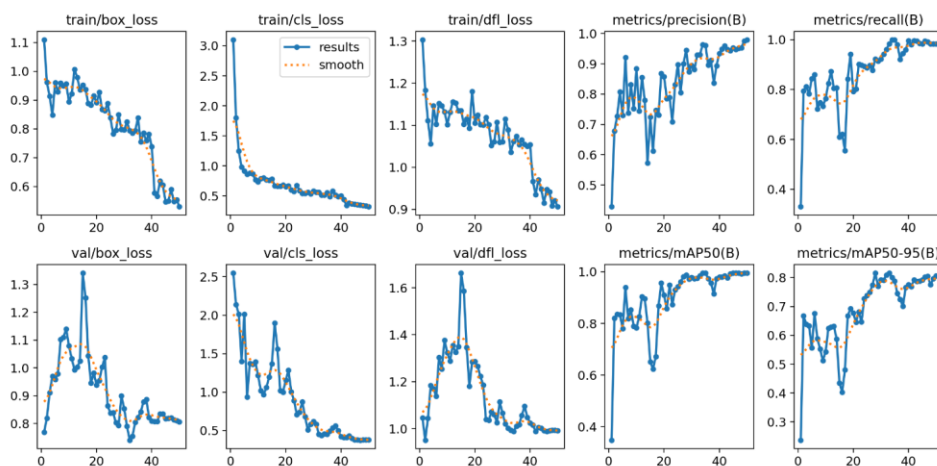


Figure 2.4.7

Across figures 2.4.1 to 2.4.6, it can be said that the model is able to perform its best at a confidence level of 0.633 (F1 Curve), and that at a lower confidence range (0.2 to 0.6), the model is still able to detect the presence of objects and correctly identify the object ~80% of the time (Precision and Recall Curve). Figure 2.4.5 shows the more direct result of the correct and incorrect identifications of the validation image set, ideally for confusion matrices should show a dark blue diagonal line, indicating a perfect performance where the model is able to distinguish between the different classes without errors. However, in this case, the "Metal" and "Paper" classes perform well but will occasionally detect part of the background as the "Plastic" class.

Figure 2.4.6 represents the summary of the training set of the image dataset, for all images metal cans appear the most, followed by paper cups and plastic bottles, this show disparity between the classes in the training dataset, and can be remedied if the paper and plastic class objects appear in the dataset more often, ideally all three classes should have equal instances in the dataset, as for the top right part of figure 2.4.6, it represents all of the bounding boxes from the training image set overlayed on top of each other, a "fuzzier" look represents good variation in terms of object size and orientation. As for the bottom part of figure 2.4.6, it represents the scatter plot of object bounding box location and size, a more scattered plot means the objects in the dataset have higher location and size variance and is generally a better dataset. Figure 2.4.7 shows the model training progress, a converging graph on the "Loss" curves mean the model is better generally. **Appendix 2.4.1** and **appendix 2.4.2** shows the training and validation results.

3.0 Discussion

3.1 Stakeholder Engagement

Stakeholder input played a key role in guiding the technical aspects, real-world usage, and social & environmental consequences of the smart waste sorting proposed solution. Stakeholders within the industry view the accurate segregation of recyclable materials as a critical input to meeting recyclables' industry requirements for processing/recycling and to further ensure that accurate sorting is essential to achieve optimal results. As such, **Mohammad Ali** (CEO, PET Bottle recycling, Savar International Trading) mentioned the proposed project can play a great role in reducing manpower and that is why it is very important for the system to detect and differentiate the waste particle accurately. In addition, the academic community has also provided technical guidance by ensuring that the proposed design and construction is based upon sound engineering principles, electrical safety, and functional reliability. For instance, **Dr. Ker Pin Jern** (Professor, Faculty of Engineering and Technology, Sunway University) described how we should continue to work towards increasing accuracy in detecting waste to promote correct waste sorting behavior through education. In addition to **Associate Professor Dr. Chia Wai Chong** (Programme Leader for the Bachelor of Electronic and Electrical Engineering (Hons), School of Engineering, Sunway University) highlighted that the project will help with real waste management issues within society, he also stated the necessity of well-defined processes for validating the results.

3.2 Evaluation Against Sustainability (Environmental, Economic and Social Pillars), Safety & Health

From a sustainability perspective, there are three different pillars: Environmental, Economic, and Social. The Smart Waste Sorting System positively impacts these three pillars in the following ways: Environmentally, successful sorting of waste allows for better, more efficient recycling, less dependency on landfills, and therefore less greenhouse gas emissions due to less sorting and transporting of waste. Secondly, economically, the Smart Waste Sorting System reduces dependency upon human labor to perform sorting of waste, thereby lowering the cost of handling the waste, and increasing the value of recyclables by means of cleaner sorted products. Socially, the system serves to promote responsible behaviors related to waste disposal, as well as to educate and inform users and students about recycling. Lastly, the reduction of direct human interaction with waste due to the equipment's ability to automatically sort waste reduces the risk associated with hygiene, injury, and contamination. Electrical safety was ensured with a safe operating voltage, as well as through the proper insulation of system components. Overall, the Smart Waste Sorting System provides a safer

method for sorting waste, while contributing positively to sustainable and responsible methods of managing waste.

4.0 Conclusion

In conclusion, our smart waste sorting system effectively addresses the challenges of waste management in Sunway City. Using the CDIO framework and proper use of decision matrices, the system is perfectly aligned with the United Nations Sustainable Development Goals. The stakeholders' feedback helped in addressing ways of improving this project with suggestions for using an infrared sensor or ultrasonic sensor. Sustainability has been well-achieved across environmental, social, and economic pillars. Above all, this prototype can be useful in waste management while demonstrating scalability and enhancing sustainability.

5.0 Appendix

Appendix 2.1 Decision Matrix on selecting appropriate problems to be solved in Sunway City

		Problems	
Criteria	Weightage	Canopy Overheating	Waste Management
E1: Waste reduction / environmental mitigation potential	0.11	1	4.5
E2: Environmental footprint / impact	0.11	2.5	4
E3: Resource conservation / sustainability	0.11	1.5	4
S1: User safety / health impact	0.11	3.5	4
S2: Accessibility & inclusiveness	0.11	3.5	4.5
S3: Community acceptance / relevance	0.11	4	4.5
C1: Initial implementation cost	0.113	2.5	3.5
C2: Operating / maintenance cost	0.113	3	4
C3: ROI / value for money	0.113	2.5	4
Total (out of 5)	1	2.67	3.61

Appendix 2.2.2 Decision Matrix on selecting appropriate solutions

Commented [Y1]: Format for consistency

		Solutions			
Criteria	Weightage	Automated Waste Sorter (IoT + ML + Actuators)	Awareness Campaign + Signage	Manual On-site Sorting Facility	Passive Mechanical Sorter
E1: Waste reduction potential	0.11	5	2	3	3
E2: Environmental footprint / impact	0.11	5	2	3	3
E3: Resource conservation / circularity	0.11	4	2	3	3
S1: User safety	0.11	4	4	3	4
S2: Accessibility & inclusiveness	0.11	4	5	3	3
S3: Community acceptance	0.11	4	4	3	3
C1: Initial implementation cost	0.113	3	5	2	3
C2: Operating / maintenance cost	0.113	3	5	2	3
C3: ROI / value for money	0.113	4	3	2	3
Total (out of 5)	1	3.99	3.56	2.66	3.11

Further details on the criteria of decision matrix

Environmental (E)

- E1 Waste reduction potential — how much the intervention reduces waste/landfill.
- E2 Environmental Footprint/ impact — measurable ecological benefits (pollution, emissions).
- E3 Resource conservation / circularity — ability to recover / reuse materials.

Social (S)

- S1 User safety — reduces hazards or improves health.
- S2 Accessibility & inclusiveness — usable by broad population.
- S3 Community acceptance — likelihood of public adoption.

Economic (C)

- C1 Initial implementation cost — affordability to start (lower cost → higher score).
- C2 Operating / maintenance cost — long-term running cost (lower → higher score).
- C3 ROI / value for money — savings, recoverable value, scalability.

Appendix 2.2.3 Decision Matrix on sorting mechanism

Criteria	Weightage	Mechanisms		
		Servo Motor Hatch	Conveyor Belt System	Rotating Sweeper
Technical complexity	0.15	4	3	3
Size	0.15	5	3	4
Components Required	0.20	3	4	3
Reliability	0.25	3	5	3
Cost	0.25	4	4	3
Total (out of 5)	1	3.70	3.95	3.15

Appendix 2.2.4 Decision Matrix on chassis construction material for 3D printing

Criteria	Weightage	Chassis Construction Material for 3D printing		
		PLA	PETG HF	ABS
Cost	0.20	5	4	3
Environmental sustainability	0.15	5	4	2
Printability/Ease of Printing	0.25	5	4	2
Mechanical Strength	0.20	4	5	5
Dimension Accuracy	0.20	5	4	3
Total (out of 5)	1	4.8	4.2	3.0

Appendix 2.2.5 Decision Matrix on microcontroller selection

Criteria	Weightage	Microcontroller		
		Arduino Nano	ESP 32	Raspberry PI
Cost	0.2	5	4	2
Ease of use	0.15	5	4	3
Size	0.15	5	4	3
Core Performance	0.2	2	4	5
Features	0.2	2	5	4
Power Draw	0.1	4	4	2
Total (out of 5)	1	3.7	4.2	3.3

Appendix 2.2.6 Decision Matrix on IoT platform

Criteria	Weightage	IoT Platform		
		Blynk	ThingSpeak	Custom
User Friendliness	0.2	5	4	2
Device Control	0.2	5	3	4
Data Analytic	0.2	4	5	3
Cost	0.15	5	5	3
Ease of Implementation	0.25	5	4	2
Total (out of 5)	1	4.8	4.15	2.75

Appendix 2.2.7 Decision Matrix on prototyping circuit board selection

Criteria	Weightage	Prototyping Circuit Board Selection		
		Donut Board	Solderless Breadboard	PCB
Cost	0.15	4	5	2
Ease of Assembly	0.20	4	5	2
Flexibility for prototyping	0.15	4	5	2
Reliability/ Connection stability	0.25	5	3	5
Time to implement/ Availability	0.25	5	5	2
Total (out of 5)	1	4.5	4.5	2.75

Appendix 2.2.8 Decision Matrix on recognition intelligence of the system (sensing)

Criteria	Weightage	Recognition Intelligence of system		
		Capacitive and Inductive Sensors	Image recognition AI models (ML)	Hybrid (Sensors + ML)
Cost	0.2	5	3	2
Accuracy/ Classification Precision	0.25	3	5	5
Ease of Integration	0.2	5	5	3
Scalability/ Future Upgrades	0.2	3	5	5
Time to Implement	0.15	5	4	2
Total (out of 5)	1	4.10	4.45	3.55

Appendix 2.2.9 Decision Matrix on AI model for image recognition

Criteria	Weightage	AI model for image recognition		
		YOLO	SSD	Faster R-CNN
Accuracy/Detection Precision	0.25	5	4	5
Inference Speed/ Real Time Performance	0.25	5	4	3
Ease of Integration with system	0.2	5	4	3
Model size	0.15	5	4	3
Adaptability to custom dataset (training)	0.15	4	4	5
Total (out of 5)	1	4.85	4	3.8

Appendix 2.3.1. ARDUINO IDE Code for the ESP32:

```
//Blynk Information (MUST BE PLACE AT TOP)
#define BLYNK_TEMPLATE_ID "TMPL62tDqg4pN"
#define BLYNK_TEMPLATE_NAME "ESP SmartBin"
#define BLYNK_AUTH_TOKEN "WjtTDWeQVPBWrl7meyhXTMV3ptvVGRhk"

#include <WiFi.h>
#include <WebServer.h>
#include <ESP32Servo.h>
#include <ESP_Mail_Client.h>
#include <HTTPClient.h>
#include <BlynkSimpleEsp32.h>
#include "HX711.h"
#include <math.h>

const char* ssid = "SunwayEdu";
const char* password = "";

//Blynk Virtual Pins
#define BLYNK_MET_WEIGHT V0
#define BLYNK_PAP_WEIGHT V1
#define BLYNK_PLA_WEIGHT V2
#define BLYNK_MET_FULL V3
#define BLYNK_PAP_FULL V4
#define BLYNK_PLA_FULL V5

//Blynk Waste Tracking Variables
float dailyWeight_MET = 0;
float dailyWeight_PAP = 0;
float dailyWeight_PLA = 0;

Servo ServoLeft;
Servo ServoRight;

HX711 scaleMet;
HX711 scalePla;
HX711 scalePap;

#define SERVOPIN_LEFT 5
#define SERVOPIN_RIGHT 18

#define DOUT_MET 39
#define DOUT_PAP 34
#define DOUT_PLA 35
#define SCK 33

//Load Cell Calibration Parameters
float calibration_factor_paper = 202;
float calibration_factor_metal = 213;
float calibration_factor_plastic = 212;

//Conveyor Belt Motor
#define MOTOR_PWM 26
#define DIRECTION_PIN 27

//Bin 1 capacity status
bool bin_Met_full = false;
bool bin_Pap_full = false;
bool bin_Pla_full = false;

//Load Cell Reading Timer
unsigned long lastScaleReadTime = 0;
const unsigned long scaleReadInterval = 2000; //in (ms)

//Checkpoint LEDs
#define WiFi_LED 3
```

```

#define EMAIL_LED 17
#define PROCESSING_LED 4
#define READY_LED 16

#define SMTP_HOST "smtp.gmail.com"
#define SMTP_PORT 465

#define AUTHOR_EMAIL "INSERT AUTHOR EMAIL"
#define AUTHOR_PASSWORD "INSERT AUTHOR EMAIL PASSWORD"

#define RECIPIENT_EMAIL "INSERT RECIPIENT EMAIL HERE"
Session_Config config;

SMTPSession smtp;

//Server Port Initialization
WebServer server(80);

void smtpCallback(SMTP_Status status){
    /* Print the current status */
    Serial.println(status.info());

    /* Print the sending result */
    if (status.success()){
        // ESP_MAIL_PRINTF used in the examples is for format printing via debug Serial port
        // that works for all supported Arduino platform SDKs e.g. AVR, SAMD, ESP32 and ESP8266.
        // In ESP8266 and ESP32, you can use Serial.printf directly.

        Serial.println("-----");
        ESP_MAIL_PRINTF("Message sent success: %d\n", status.completedCount());
        ESP_MAIL_PRINTF("Message sent failed: %d\n", status.failedCount());
        Serial.println("-----\n");

        for (size_t i = 0; i < smtp.sendingResult.size(); i++)
        {
            /* Get the result item */
            SMTP_Result result = smtp.sendingResult.getItem(i);

            // In case, ESP32, ESP8266 and SAMD device, the timestamp get from result.timestamp should be valid if
            // your device time was synched with NTP server.
            // Other devices may show invalid timestamp as the device time was not set i.e. it will show Jan 1, 1970.
            // You can call smtp.setSystemTime(xxx) to set device time manually. Where xxx is timestamp (seconds since Jan 1,
            1970)

            ESP_MAIL_PRINTF("Message No: %d\n", i + 1);
            ESP_MAIL_PRINTF("Status: %s\n", result.completed ? "success" : "failed");
            ESP_MAIL_PRINTF("Date/Time: %s\n", MailClient.Time.getDateTimeString(result.timestamp,
            "%B %d, %Y %H:%M:%S").c_str());
            ESP_MAIL_PRINTF("Recipient: %s\n", result.recipients.c_str());
            ESP_MAIL_PRINTF("Subject: %s\n", result.subject.c_str());
        }
        Serial.println("-----\n");

        // You need to clear sending result as the memory usage will grow up.
        smtp.sendingResult.clear();
    }
}

void emailbin_Met() {
    SMTP_Message message;

    message.sender.name = F("ESP");
    message.sender.email = AUTHOR_EMAIL;
    message.subject = F("ESP Test Email");
    message.addRecipient(F("Dong"), RECIPIENT_EMAIL);

    String textMsg = "Metal Rubbish Bin located at B1, FET Building is Full";

```

```

message.text.content = textMsg.c_str();
message.text.charset = "us-ascii";
message.text.transfer_encoding = Content_Transfer_Encoding::enc_7bit;

message.priority = esp_mail_smtp_priority::esp_mail_smtp_priority_low;
message.response.notify = esp_mail_smtp_notify_success | esp_mail_smtp_notify_failure | esp_mail_smtp_notify_delay;

if (!MailClient.sendMail(&smtp, &message, false)) {
    ESP_MAIL_PRINTF("Error, Status Code: %d, Error Code: %d, Reason: %s", smtp.statusCode(), smtp.errorCode(),
smtp.errorReason().c_str());
}
}

void emailbin_Pap() {
    SMTP_Message message;

    message.sender.name = F("ESP");
    message.sender.email = AUTHOR_EMAIL;
    message.subject = F("ESP Test Email");
    message.addRecipient(F("Dong"), RECIPIENT_EMAIL);

    String textMsg = "Paper Rubbish Bin located at B1, FET Building is Full";
    message.text.content = textMsg.c_str();
    message.text.charset = "us-ascii";
    message.text.transfer_encoding = Content_Transfer_Encoding::enc_7bit;

    message.priority = esp_mail_smtp_priority::esp_mail_smtp_priority_low;
    message.response.notify = esp_mail_smtp_notify_success | esp_mail_smtp_notify_failure | esp_mail_smtp_notify_delay;

    if (!MailClient.sendMail(&smtp, &message, false)) {
        ESP_MAIL_PRINTF("Error, Status Code: %d, Error Code: %d, Reason: %s", smtp.statusCode(), smtp.errorCode(),
smtp.errorReason().c_str());
    }
}

void emailbin_Pla() {
    SMTP_Message message;

    message.sender.name = F("ESP");
    message.sender.email = AUTHOR_EMAIL;
    message.subject = F("ESP Test Email");
    message.addRecipient(F("Dong"), RECIPIENT_EMAIL);

    String textMsg = "Plastic Rubbish Bin located at B1, FET Building is Full";
    message.text.content = textMsg.c_str();
    message.text.charset = "us-ascii";
    message.text.transfer_encoding = Content_Transfer_Encoding::enc_7bit;

    message.priority = esp_mail_smtp_priority::esp_mail_smtp_priority_low;
    message.response.notify = esp_mail_smtp_notify_success | esp_mail_smtp_notify_failure | esp_mail_smtp_notify_delay;

    if (!MailClient.sendMail(&smtp, &message, false)) {
        ESP_MAIL_PRINTF("Error, Status Code: %d, Error Code: %d, Reason: %s", smtp.statusCode(), smtp.errorCode(),
smtp.errorReason().c_str());
    }
}

void smtpCallback(SMTP_Status status);

void handleRoot() {
    server.send(200, "text/plain", "ESP32 alive");
}

void handleTrigger() {
    Serial.println("Trigger received!");
    digitalWrite(2, HIGH);
    delay(200);
}

```

```

digitalWrite(2, LOW);
server.send(200, "text/plain", "OK");
}

float readprintWeight(HX711& scale) {
  if(scale.is_ready()) {
    return scale.get_units(1);
  }

  else {
    Serial.println("HX711 not found.");
    return 0;
  }
}

void handlePaper() {

  // 1. Log Detection
  // Note: The original code incorrectly printed "Metal Can Detected" for all materials.
  Serial.print("Paper");
  Serial.println(" Detected");

  float baseline = readprintWeight(scalePap);
  float threshold = 15;
  float diff = 0;

  // 2. Initial Action (Flash LED & Open Gate)
  digitalWrite(2, HIGH);
  ServoLeft.write(180);
  ServoRight.write(135);
  digitalWrite(READY_LED, LOW);
  delay(500);

  // 3. Wait for Object to Drop (Weight Confirmation Loop)
  Serial.println("Waiting for object to drop...");
  while (true) {
    Blynk.run();
    digitalWrite(PROCESSING_LED, HIGH);
    float current = readprintWeight(scalePap);
    diff = current - baseline;
    Serial.print("Baseline: ");
    Serial.print(baseline);
    Serial.print(" Current: ");
    Serial.print(current);
    Serial.print(" Diff: ");
    Serial.println(diff);
    digitalWrite(PROCESSING_LED, LOW);

    // Conveyor Belt Activation (Running during the drop confirmation)
    digitalWrite(MOTOR_PWM, HIGH);
    digitalWrite(DIRECTION_PIN, HIGH);

    if (diff >= threshold) {
      Serial.println("Object confirmed in bin!");
      break;
    }
    delay(300);
  }

  // 4. Final Action (Reset Hardware & Send Response)
  dailyWeight_PAP += diff;
  digitalWrite(2, LOW);
  digitalWrite(PROCESSING_LED, LOW);
  ServoLeft.write(90);
  ServoRight.write(90);
  digitalWrite(MOTOR_PWM, LOW);
  digitalWrite(DIRECTION_PIN, LOW);
  Blynk.virtualWrite(BLYNK_PAP_WEIGHT, dailyWeight_PAP);
}

```

```

Serial.print("Accumulated Paper Weight: ");
Serial.println(dailyWeight_PAP);

// Send HTTP response back to the client/YOLO system
server.send(200, "text/plain", "Paper Sorted");
}

void handlePlastic() {

// 1. Log Detection
// Note: The original code incorrectly printed "Metal Can Detected" for all materials.
Serial.print("Paper");
Serial.println(" Detected");

float baseline = readprintWeight(scalePla);
float threshold = 8;
float diff = 0;

// 2. Initial Action (Flash LED & Open Gate)
digitalWrite(2, HIGH);
ServoLeft.write(45);
ServoRight.write(0);
digitalWrite(READY_LED, LOW);
delay(500);

// 3. Wait for Object to Drop (Weight Confirmation Loop)
Serial.println("Waiting for object to drop...");
while (true) {
  Blynk.run();
  digitalWrite(PROCESSING_LED, HIGH);
  float current = readprintWeight(scalePla);
  diff = current - baseline;
  Serial.print("Baseline: ");
  Serial.print(baseline);
  Serial.print(" Current: ");
  Serial.print(current);
  Serial.print(" Diff: ");
  Serial.println(diff);
  digitalWrite(PROCESSING_LED, LOW);

// Conveyor Belt Activation (Running during the drop confirmation)
digitalWrite(MOTOR_PWM, HIGH);
digitalWrite(DIRECTION_PIN, HIGH);

  if (diff >= threshold) {
    Serial.println("Object confirmed in bin!");
    break;
  }
  delay(300);
}

// 4. Final Action (Reset Hardware & Send Response)
dailyWeight_PLA += diff;
digitalWrite(2, LOW);
digitalWrite(PROCESSING_LED, LOW);
ServoLeft.write(90);
ServoRight.write(90);
digitalWrite(MOTOR_PWM, LOW);
digitalWrite(DIRECTION_PIN, LOW);
Blynk.virtualWrite(BLYNK_PLA_WEIGHT, dailyWeight_PLA);
Serial.print("Accumulated Plastic Weight: ");
Serial.println(dailyWeight_PLA);

// Send HTTP response back to the client/YOLO system
server.send(200, "text/plain", "Plastic Sorted");
}

void handleMetal() {

```

```
// 1. Log Detection
// Note: The original code incorrectly printed "Metal Can Detected" for all materials.
Serial.print("Metal");
Serial.println(" Detected");
```

```
float baseline = readprintWeight(scaleMet);
float threshold = 10;
float diff = 0;
```

```
// 2. Initial Action (Flash LED & Open Gate)
digitalWrite(2, HIGH);
ServoLeft.write(85);
ServoRight.write(90);
digitalWrite(READY_LED, LOW);
delay(500);
```

```
// 3. Wait for Object to Drop (Weight Confirmation Loop)
Serial.println("Waiting for object to drop...");
while (true) {
  Blynk.run();
  digitalWrite(PROCESSING_LED, HIGH);
  float current = readprintWeight(scaleMet);
  diff = current - baseline;
  Serial.print("Baseline: ");
  Serial.print(baseline);
  Serial.print(" Current: ");
  Serial.print(current);
  Serial.print(" Diff: ");
  Serial.println(diff);
  digitalWrite(PROCESSING_LED, LOW);

  // Conveyor Belt Activation (Running during the drop confirmation)
  digitalWrite(MOTOR_PWM, HIGH);
  digitalWrite(DIRECTION_PIN, HIGH);

  if (diff >= threshold) {
    Serial.println("Object confirmed in bin!");
    break;
  }
  delay(300);
}
```

```
// 4. Final Action (Reset Hardware & Send Response)
dailyWeight_MET += diff;
digitalWrite(2, LOW);
digitalWrite(PROCESSING_LED, LOW);
ServoLeft.write(90);
ServoRight.write(90);
digitalWrite(MOTOR_PWM, LOW);
digitalWrite(DIRECTION_PIN, LOW);
Blynk.virtualWrite(BLYNK_MET_WEIGHT, dailyWeight_MET);
Serial.print("Accumulated Metal Weight: ");
Serial.println(dailyWeight_MET);
```

```
// Send HTTP response back to the client/YOLO system
server.send(200, "text/plain", "Metal Sorted");
}
```

```
void handleMulti() {
  Serial.println("Multiple Objects Detected, Waiting for Removal");
}
```

```
void setup(){
  //initializing serial monitor
  Serial.begin(115200);

  //Initializing Checkpoint LEDs
```

```

pinMode(WiFi_LED, OUTPUT);
pinMode(EMAIL_LED, OUTPUT);
pinMode(PROCESSING_LED, OUTPUT);
pinMode(READY_LED, OUTPUT);

//initializing individual load cells
scaleMet.begin(DOUT_MET, SCK);
delay(500);
scalePap.begin(DOUT_PAP, SCK);
delay(500);
scalePla.begin(DOUT_PLA, SCK);
delay(500);

delay(500);
if(scaleMet.is_ready()){
  Serial.println("Metal Scale Is Ready");
} else if(!scaleMet.is_ready()){
  Serial.println("Metal Scale Not Working");
}

delay(500);
if(scalePap.is_ready()){
  Serial.println("Paper Scale is Ready");
} else if(!scalePap.is_ready()){
  Serial.println("Paper Scale Not Working");
}

delay(500);
if(scalePla.is_ready()){
  Serial.println("Plastic Scale is Ready");
} else if(!scalePla.is_ready()){
  Serial.println("Plastic Scale Not Working");
}

//Set 0 offset for all load cells
delay(500);
scaleMet.tare();
delay(500);
scalePap.tare();
delay(500);
scalePla.tare();

//Calibrate load cells according to factor
delay(500);
scaleMet.set_scale(calibration_factor_metal);
delay(500);
scalePap.set_scale(calibration_factor_paper);
delay(500);
scalePla.set_scale(calibration_factor_plastic);
Serial.println("Load Cells Initialized and Calibrated");
Serial.print("Metal Bin Current Reading: ");
Serial.println(scaleMet.get_units(5));
Serial.print("Paper Bin Current Reading: ");
Serial.println(scalePap.get_units(5));
Serial.print("Plastic Bin Current Reading: ");
Serial.println(scalePla.get_units(5));

///Attaching Servo
ServoLeft.attach(SERVOPIN_LEFT);
ServoRight.attach(SERVOPIN_RIGHT);
ServoLeft.write(90);
ServoRight.write(90);
Serial.println("Servos Initialized and Attached");

//Resetting Conveyor Belt
pinMode(DIRECTION_PIN, OUTPUT);
digitalWrite(DIRECTION_PIN, LOW);

```

```

//Onboard LED Reset
pinMode(2, OUTPUT);
digitalWrite(2, LOW);

//Initializing WiFi
WiFi.begin(ssid, password);
Serial.println("Connecting to WiFi");

//Connecting to WIFI
while (WiFi.status() != WL_CONNECTED) {
  digitalWrite(WiFi_LED, LOW);
  delay(200);
  Serial.print(".");
  digitalWrite(WiFi_LED, HIGH);
  delay(200);
}

//Confirm WIFI Connection
Serial.println();
Serial.print("Connected, IP: ");
Serial.println(WiFi.localIP());
//Wifi LED connection indicator
digitalWrite(WiFi_LED, HIGH);

//Converting the IP Address into a String
String localIP = WiFi.localIP().toString();

//Webserver Trigger Handling
server.on("/", handleRoot);
//server.on("/trigger", handleTrigger);
server.on("/Paper", handlePaper);
server.on("/Metal", handleMetal);
server.on("/Plastic", handlePlastic);
server.on("/Multi", handleMulti);
server.begin();
Serial.println("Server Setup Complete");

//Starting Email Server
config.server.host_name = "smtp.gmail.com";
config.server.port = 587;

config.login.email = AUTHOR_EMAIL;
config.login.password = AUTHOR_PASSWORD;
config.login.user_domain = "";

config.time.ntp_server = "pool.ntp.org";
config.time.gmt_offset = 8;
config.time.day_light_offset = 0;

smtp.debug(0);
smtp.connect(&config);
Serial.println("Initiating Email Server");
smtp.callback(smtpCallback);
while (!smtp.connected()){
  digitalWrite(EMAIL_LED, HIGH);
  delay(200);
  digitalWrite(EMAIL_LED, LOW);
  Serial.print(".");
  delay(200);
}
Serial.println("Email Server Initiated");
//Email Connection LED Indicator
digitalWrite(EMAIL_LED, HIGH);

//Send IP to email address

```

```

SMTP Message message_IP;
Serial.println("Sending IP Address to Email");
message_IP.sender.name = F("ESP");
message_IP.sender.email = AUTHOR_EMAIL;
message_IP.subject = F("ESP Test Email");
message_IP.addRecipient(F("Dong"), RECIPIENT_EMAIL);

String textMsg = "ESP32 IP: " + localIP;
message_IP.text.content = textMsg.c_str();
message_IP.text.charset = "us-ascii";
message_IP.text.transfer_encoding = Content_Transfer_Encoding::enc_7bit;

message_IP.priority = esp_mail_smtp_priority::esp_mail_smtp_priority_low;
message_IP.response.notify = esp_mail_smtp_notify_success | esp_mail_smtp_notify_failure | esp_mail_smtp_notify_delay;

if (!MailClient.sendMail(&smtp, &message_IP, false)) {
    ESP_MAIL_PRINTF("Error, Status Code: %d, Error Code: %d, Reason: %s", smtp.statusCode(), smtp.errorCode(),
smtp.errorReason().c_str());
}
Serial.println("IP Address Sent To Email");

//Blynk Initialization
Blynk.begin(BLYNK_AUTH_TOKEN, ssid, password);
//Blynk Test
Blynk.virtualWrite(BLYNK_MET_WEIGHT, 0);
delay(2000);
Blynk.virtualWrite(BLYNK_PAP_WEIGHT, 0);
delay(2000);
Blynk.virtualWrite(BLYNK_PLA_WEIGHT, 0);
delay(2000);
Blynk.virtualWrite(BLYNK_MET_FULL, 0);
delay(2000);
Blynk.virtualWrite(BLYNK_PAP_FULL, 0);
delay(2000);
Blynk.virtualWrite(BLYNK_PLA_FULL, 0);

//Conveyor Motor Initialization
pinMode(DIRECTION_PIN, OUTPUT);
pinMode(MOTOR_PWM, OUTPUT);
Serial.println("Conveyor Motors Initilized");
}

void loop() {
    // PRIORITY 1: Always handle the web server first and as fast as possible
    server.handleClient();

    //Running Blynk to prevent disconnect
    Blynk.run();

    //Ready Status LED Indicator
    digitalWrite(READY_LED, HIGH);

    // PRIORITY 2: Only read scales and check emails every 2 seconds
    if (millis() - lastScaleReadTime >= scaleReadInterval) {
        lastScaleReadTime = millis();

        // Use get_units(1) for speed within the interval
        long Real_Time_Met = scaleMet.get_units(1);
        long Real_Time_Pap = scalePap.get_units(1);
        long Real_Time_Pla = scalePla.get_units(1);

        Serial.print("Paper Bin Weight: ");
        Serial.print(Real_Time_Pap);
        Serial.print(" | Metal Bin Weight: ");
        Serial.print(Real_Time_Met);
        Serial.print(" | Plastic Bin Weight: ");

```

```

Serial.println(Real_Time_Pla);

//Wifi Connection Check
if(WiFi.status() != WL_CONNECTED){
  WiFi.begin(ssid, password);
  digitalWrite(READY_LED, LOW);

  while (WiFi.status() != WL_CONNECTED) {
    digitalWrite(WiFi_LED, LOW);
    delay(200);
    Serial.print(".");
    digitalWrite(WiFi_LED, HIGH);
    delay(200);
  }

  digitalWrite(READY_LED, HIGH);
  digitalWrite(WiFi_LED, HIGH);
}

//Email Status Check

// --- Metal Full Bin Check ---
if (Real_Time_Met > 1500 && !bin_Met_full) {
  Blynk.virtualWrite(BLYNK_MET_FULL, 255);
  emailbin_Met();
  bin_Met_full = true;
} else if (Real_Time_Met < 50 && bin_Met_full) {
  Blynk.virtualWrite(BLYNK_MET_FULL, 0);
  bin_Met_full = false;
}

// --- Paper Full Bin Check ---
if (Real_Time_Pap > 1500 && !bin_Pap_full) {
  Blynk.virtualWrite(BLYNK_PAP_FULL, 255);
  emailbin_Pap();
  bin_Pap_full = true;
} else if (Real_Time_Pap < 50 && bin_Pap_full) {
  Blynk.virtualWrite(BLYNK_PAP_FULL, 0);
  bin_Pap_full = false;
}

// --- Plastic Full Bin Check ---
if (Real_Time_Pla > 1500 && !bin_Pla_full) {
  Blynk.virtualWrite(BLYNK_PLA_FULL, 255);
  emailbin_Pla();
  bin_Pla_full = true;
} else if (Real_Time_Pla < 50 && bin_Pla_full) {
  Blynk.virtualWrite(BLYNK_PLA_FULL, 0);
  bin_Pla_full = false;
}
}
}

```

```

import warnings
warnings.filterwarnings("ignore", category=RuntimeWarning) # hide numpy warnings

import cv2
from ultralytics import YOLO
import requests
import threading
import time

# ----- ASYNC REQUEST SENDER -----
def send_request_async(url):
    def task():
        try:
            requests.get(url, timeout=0.5) # short timeout
        except:
            print("Warning: Could not reach ESP32")
            threading.Thread(target=task, daemon=True).start()
# -----

# ----- CONFIG -----
MODEL_PATH = "my_model.pt" # the model file, must be in the same folder as this file
TARGET_CLASSES = ["Paper", "Metal", "Plastic"] # materials to detect
ESP32_IP = "172.18.85.248" # ESP32 IP, Check For IP Using ESP32 Code to Email, SunwayEdu IP changes everyday
SEND_TO_ESP32 = True # True to send HTTP request, False for local operation only
CAMERA_INDEX = 0 # 0 for webcam, 1,2 or 3 for external cameras
CONFIDENCE_THRESHOLD = 0.63 # Confidence percentage required before being flagged as detected

# ----- PRESET TRIGGERS -----
TRIGGER_PAPER = False
TRIGGER_METAL = False
TRIGGER_PLASTIC = False
TRIGGER_MULTI = False

# ----- CONTINUOUS DETECTION TIMERS -----
last_seen_time = {
    "Paper": 0,
    "Metal": 0,
    "Plastic": 0,
    "Multiple Objects": 0
}
currently_detected = {
    "Paper": False,
    "Metal": False,
    "Plastic": False,
    "Multiple Objects": False
}
DETECTION_HOLD_TIME = 1 # seconds required before sending trigger

# ----- LOAD MODEL -----
print("Loading YOLOv11s model...")
model = YOLO(MODEL_PATH)
model.to("cuda")
print("Model loaded!")
print("All classes in the model:", model.names)

# ----- START WEBCAM -----
cap = cv2.VideoCapture(CAMERA_INDEX)
if not cap.isOpened():
    print("Error: Could not open webcam")
    exit()

print("Starting webcam... Press 'q' to quit")

while True:
    ret, frame = cap.read()
    if not ret:
        break

```

```

results = model(frame, verbose=False)
detected_any = False # <--- NEW FLAG
multi_count = 0

for result in results:
    boxes = result.boxes
    for box in boxes:
        detected_any = True

        # Counts how many "boxes" are there
        multi_count += 1

        cls_id = int(box.cls[0])
        conf = float(box.conf[0])
        class_name = model.names[cls_id]

        if conf < CONFIDENCE_THRESHOLD:
            continue

        print(f'Detected: {class_name} ({conf:.2f})')

        # Draw box on frame
        x1, y1, x2, y2 = map(int, box.xyxy[0])
        cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
        cv2.putText(frame, f'{class_name} {conf:.2f}', (x1, y1 - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

        # --- 1-SECOND CONTINUOUS DETECTION LOGIC (NEW) ---
        if class_name in TARGET_CLASSES:

            if multi_count > 1:
                currently_detected["Multiple Objects"] = True
                currently_detected["Paper"] = False
                currently_detected["Metal"] = False
                currently_detected["Plastic"] = False

            if last_seen_time["Multiple Objects"] == 0 and last_seen_time["Paper"] == 0 and last_seen_time["Metal"] == 0
and last_seen_time["Plastic"] == 0:
                last_seen_time["Multiple Objects"] = time.time()
                if last_seen_time["Multiple Objects"] >= DETECTION_HOLD_TIME and not TRIGGER_MULTI:
                    url = f'http://{ESP32_IP}/Multi'
                    send_request_async(url)
                    print("Sent trigger to ESP32 for multiple objects")
                    TRIGGER_MULTI = True

            else:
                last_seen_time["Metal"] = 0
                last_seen_time["Plastic"] = 0
                last_seen_time["Paper"] = 0
                elapsed_multi = time.time() - last_seen_time["Multiple Objects"]
                if elapsed_multi >= DETECTION_HOLD_TIME and not TRIGGER_MULTI:
                    url = f'http://{ESP32_IP}/Multi'
                    send_request_async(url)
                    print("Sent trigger to ESP32 for multiple objects")
                    TRIGGER_MULTI = True
                continue

            if not currently_detected[class_name]:
                currently_detected[class_name] = True
                last_seen_time[class_name] = time.time()

            else:
                elapsed = time.time() - last_seen_time[class_name]

        # --- METAL TRIGGER ---
        if (class_name == "Metal"

```

```

and not TRIGGER METAL
and elapsed >= DETECTION_HOLD_TIME and multi_count == 1 and SEND_TO_ESP32):

    url = f"http://{ESP32_IP}/Metal"
    send_request_async(url)
    print(f"Sent trigger to ESP32 for {class_name}")
    TRIGGER_METAL = True

# ---- PAPER TRIGGER ----
if (class_name == "Paper"
and not TRIGGER_PAPER
and elapsed >= DETECTION_HOLD_TIME and multi_count == 1 and SEND_TO_ESP32):

    url = f"http://{ESP32_IP}/Paper"
    send_request_async(url)
    print(f"Sent trigger to ESP32 for {class_name}")
    TRIGGER_PAPER = True

# ---- PLASTIC TRIGGER ----
if (class_name == "Plastic"
and not TRIGGER_PLASTIC
and elapsed >= DETECTION_HOLD_TIME and multi_count == 1 and SEND_TO_ESP32):

    url = f"http://{ESP32_IP}/Plastic"
    send_request_async(url)
    print(f"Sent trigger to ESP32 for {class_name}")
    TRIGGER_PLASTIC = True

# ----- RESET TRIGGERS IF NO OBJECT FOUND -----
if not detected_any:
    TRIGGER_PAPER = False
    TRIGGER_METAL = False
    TRIGGER_PLASTIC = False
    TRIGGER_MULTI = False

# Reset continuous detection states
currently_detected["Paper"] = False
currently_detected["Metal"] = False
currently_detected["Plastic"] = False
currently_detected["Multiple Objects"] = False

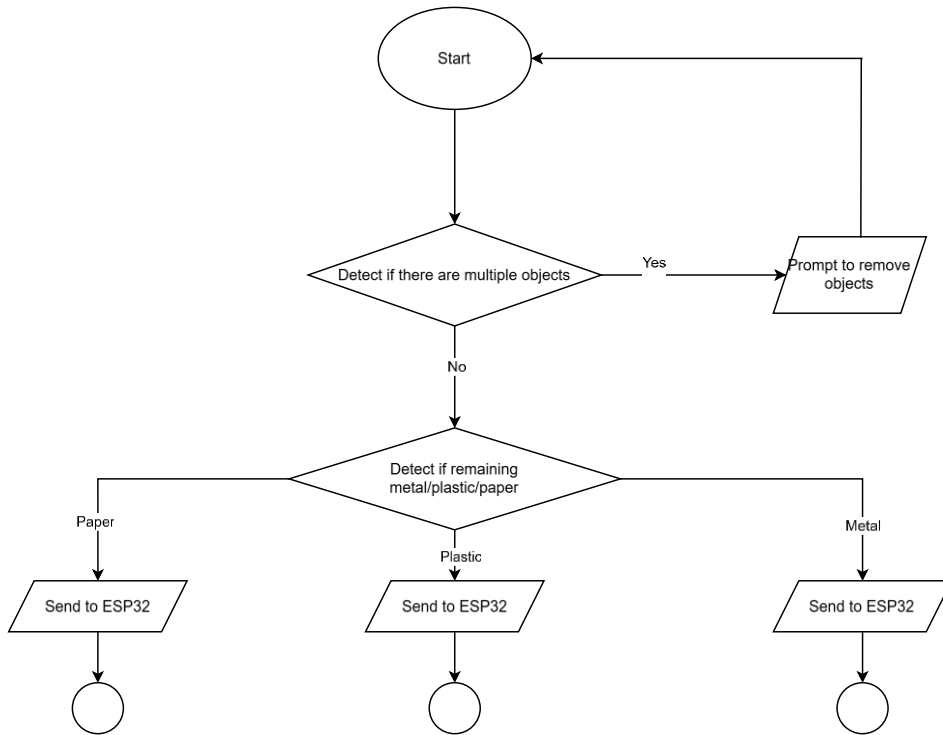
cv2.imshow("YOLO Live", frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

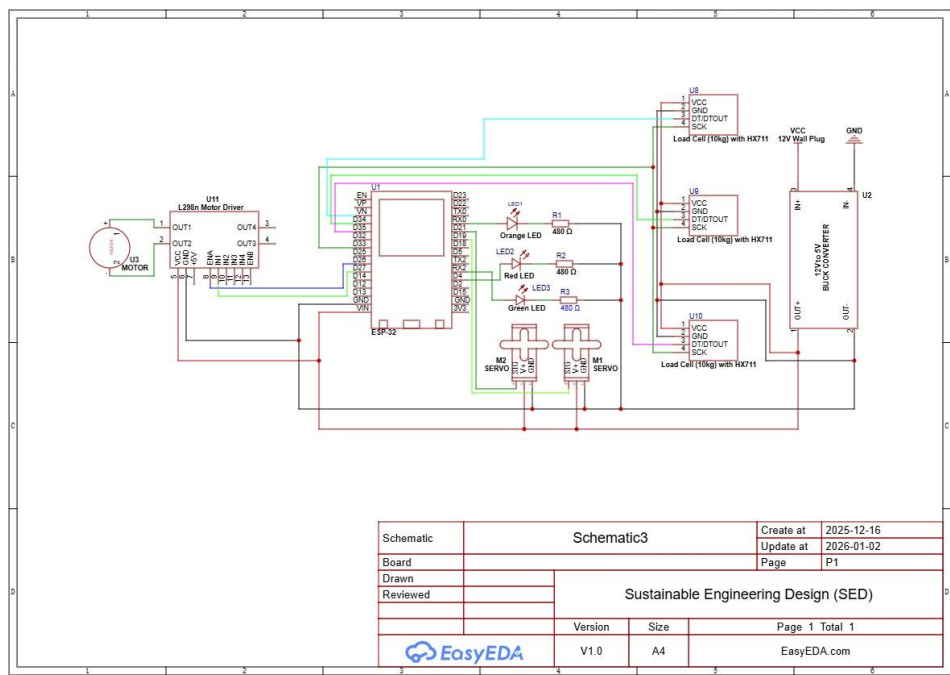
cap.release()
cv2.destroyAllWindows()

```

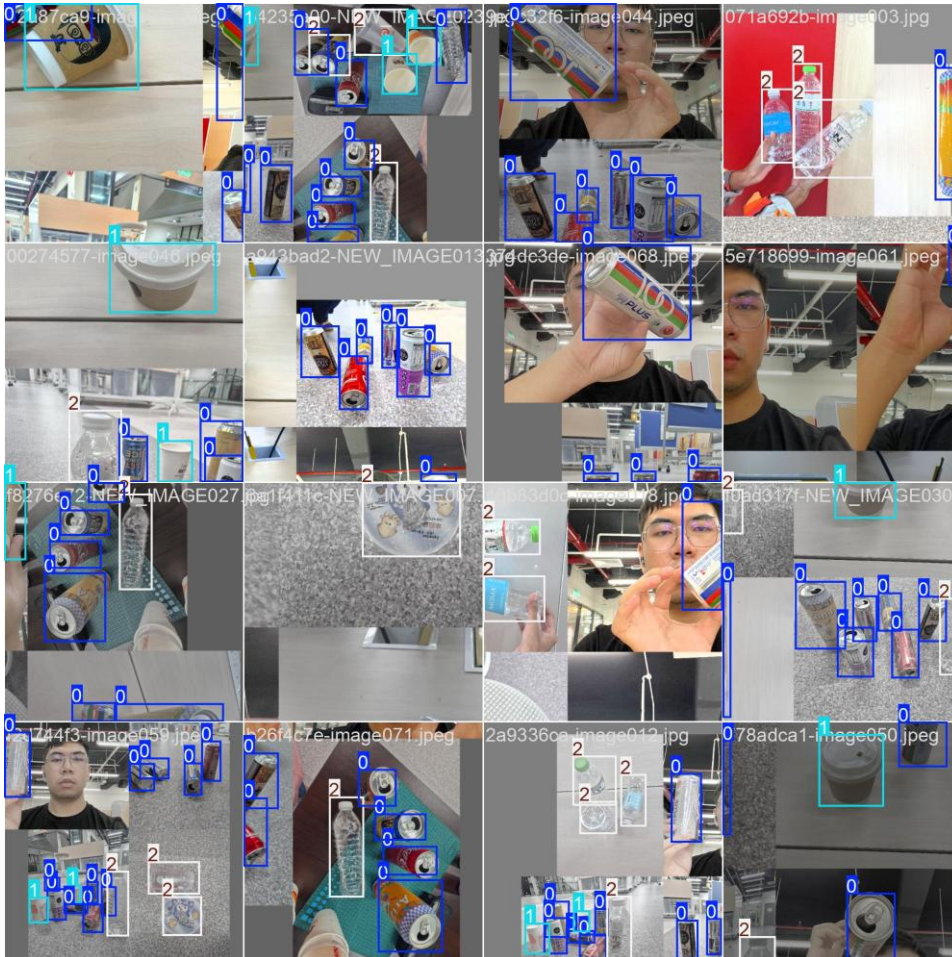
Appendix 2.3.3. Logic Flow Chart



Appendix 2.3.4. Circuit Diagram



Appendix 2.4.1. YOLO Model Training Result



Appendix 2.4.2. YOLO Model Validation (Prediction) Result

